
CASE STUDY / 2026

Smaxtec

Livestock Intelligence — sensor-to-prediction pipeline for dairy herds.

Kafka • Airflow • BigQuery • scikit-learn — Data engineering & applied ML

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	Pipeline Architecture	Page 4
03	Feature Engineering Pipeline	Page 5
04	Technology Stack	Page 6
05	The Challenge	Page 7
06	The Approach & Anomaly Flags	Page 8
07	Code Spotlight	Page 9
08	Machine Learning Models	Page 10
09	Outcome & Engagement	Page 11

This document presents an in-depth look at the Smaxtec pipeline — the data flow, the feature engineering layer, the three production models, and the real SQL that powers it all. Every diagram, code snippet, and technical claim is grounded directly in the project source.

SECTION 01

Project Overview

What Smaxtec does, the inputs it consumes, and the signals it produces.

Smaxtec sells in-stomach boluses that stream physiological telemetry from individual dairy cows — temperature, heart rate, activity level, rumination minutes, and milk yield. The platform built on top of that hardware turns the raw firehose into three actionable signals per cow: a current **health status**, a **fertility likelihood**, and an expected **milk-yield forecast** — every signal grounded in herd-wide context, not fixed thresholds.

Under the hood the system is a classical streaming-to-ML pipeline: Kafka ingests the sensor events, Airflow orchestrates the run, BigQuery stores the raw stream and computes the engineered features through a single chained SQL CTE, and three scikit-learn models — versioned as joblib artifacts inside the DAG tree — consume those features. The whole thing refreshes on schedule, so a farm operator can see deteriorating cow health within hours rather than days.

The design principle is straightforward: every model trains and scores from the *same* feature table. There is no second feature pipeline at serving time, which means no drift between what the model learned and what it sees in production.



SECTION 02

Pipeline Architecture

Sensor stream lands in BigQuery, gets engineered into features, then feeds three models.

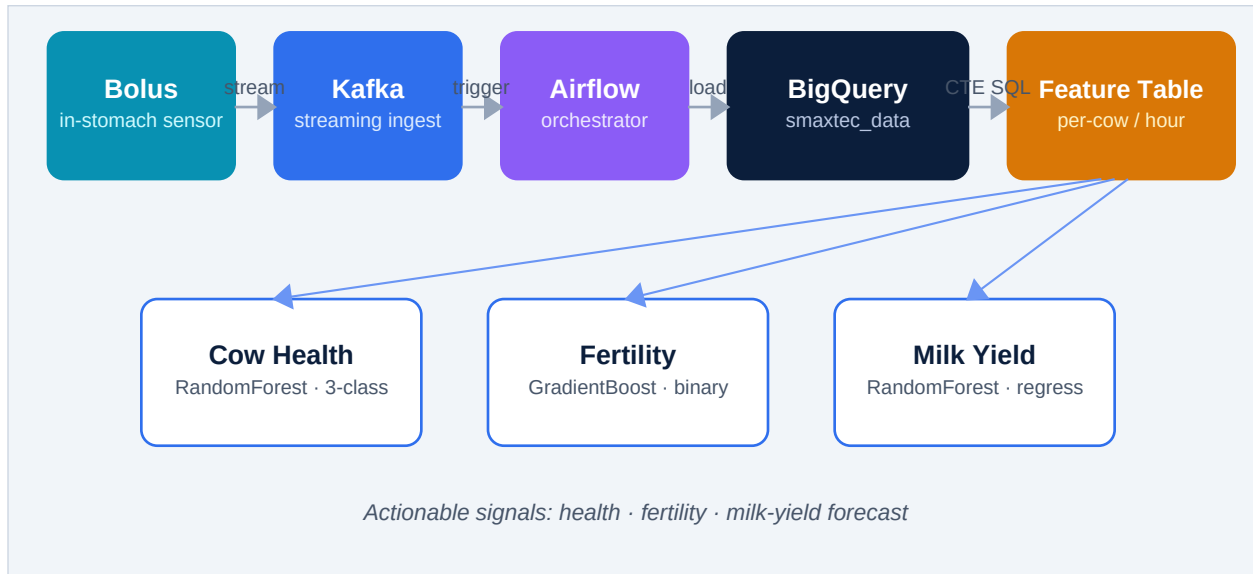


Figure — Bolus → Kafka → Airflow → BigQuery → engineered feature table → three sklearn models.

Each layer has a single, well-defined job. **Kafka** buffers and decouples the high-frequency sensor stream from downstream processing. **Airflow** orchestrates the BigQuery jobs and the model training/scoring steps. **BigQuery** holds both raw events and engineered features in `kafka-airflow-data-project.smaxtec_dataset.smaxtec_data`, with the engineered features computed by a single multi-CTE SQL script. **scikit-learn** models — Random Forest, Gradient Boosting — read directly from the same feature table.

SECTION 03

Feature Engineering Pipeline

One SQL script computes every aggregate, benchmark, flag, and label used by all three models.

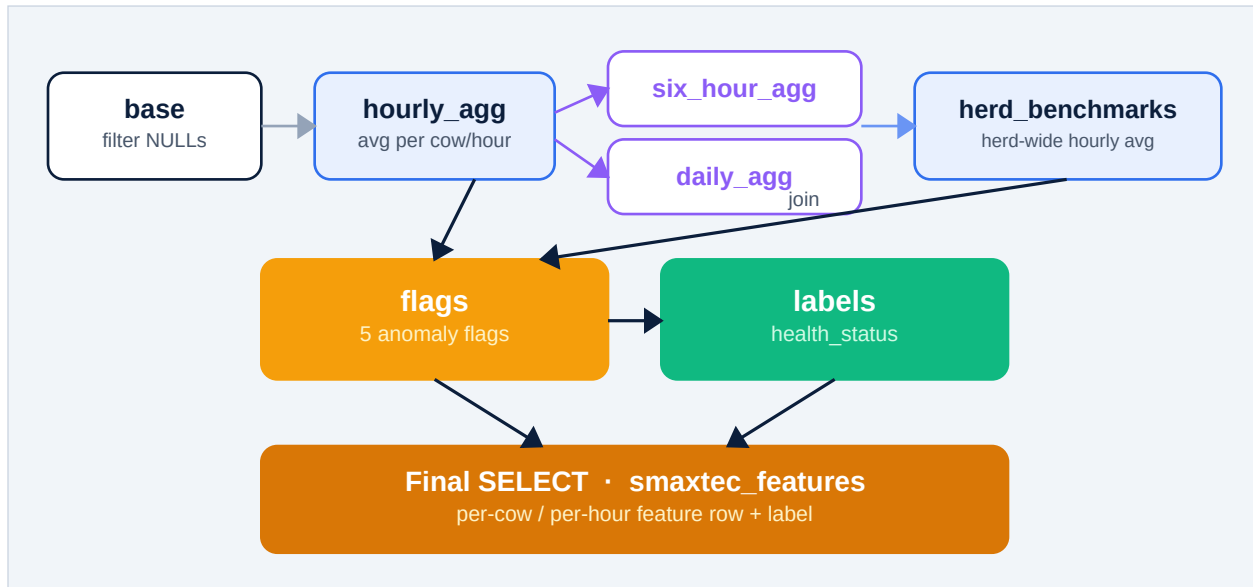


Figure — CTE chain — *base* → *hourly_agg* → (*six_hour_agg*, *daily_agg*) and *herd_benchmarks* → *flags* → *labels*.

The pipeline starts in **base**, where rows with any NULL in temperature, humidity, heart rate, rumination, milk yield, activity, or timestamp are filtered out. **hourly_agg** rolls the cleaned stream up to per-cow / per-hour averages; **six_hour_agg** and **daily_agg** produce coarser grains for trend features. **herd_benchmarks** aggregates the hourly rows across the whole herd so each cow can be compared against her peers. **flags** joins each hourly row against the herd benchmark and derives five domain anomaly flags; **labels** resolves the flags into a three-class health label. The final SELECT produces *smaxtec_features* — one row per cow per hour, ready for ML.

SECTION 04

Technology Stack

Streaming ingest, warehouse-side feature engineering, classical ML, all orchestrated by Airflow.

Layer	Technology & Details
Ingest	Apache Kafka — high-throughput event stream from bolus telemetry
Orchestrator	Apache Airflow — DAGs and model artifacts colocated under <i>airflow/dags/</i>
Warehouse	Google BigQuery — project <i>kafka-airflow-data-project</i> , dataset <i>smaxtec_dataset</i>
Feature SQL	Chained CTE: <i>base</i> → <i>hourly_agg</i> → <i>six_hour_agg</i> → <i>daily_agg</i> → <i>herd_benchmarks</i> → <i>flags</i> → <i>labels</i>
ML	scikit-learn 1.6.1 — RandomForestClassifier, GradientBoostingClassifier, RandomForestRegressor
Persistence	joblib — three production models versioned next to the DAG (<i>cow_health</i> , <i>fertility</i> , <i>milk_yield</i>)
Window Logic	BigQuery <i>LAG OVER (PARTITION BY cow_id ORDER BY hour_ts)</i> for hour-over-hour detection
Schedule	Airflow-driven recurring refresh of features and re-scoring of all three models

Why this stack. BigQuery makes feature engineering set-based and trivially parallelisable; rolling herd-wide averages and window-function-based change detection are single-pass SQL, not Python loops. Airflow keeps the lineage explicit and replayable. scikit-learn's joblib persistence keeps model artifacts portable and reviewable.

SECTION 05

The Challenge

Raw bolus telemetry is meaningless until contextualised against the rest of the herd.

Bolus telemetry arrives at a fast cadence, is noisy, and on its own tells you very little. A heart rate of 88 bpm is normal on its own; a 20-bpm drop in an hour is not. A rumination level that looks fine in absolute terms can be a warning sign if it is twenty percent below the rest of the herd. Operators need decisions, not raw numbers — but every threshold that looks sensible in isolation falls apart across seasons, breeds, and stress events.

The pipeline therefore had to do four things at once: ingest the raw stream reliably; reduce it into biologically meaningful features at hour, six-hour and daily grains; benchmark every cow against the herd in real time; and feed three separate ML models without train/serve skew. The hardest constraint was that every model — health, fertility, milk yield — has to consume the *same* engineered feature definition the warehouse computes, so a model trained on history scores correctly on fresh data.

Operationally the brief came with a fifth requirement: detect *sudden* changes, not just absolute outliers. A clinical illness shows up first as a step-change in vital signs hours before it shows up in any single threshold being crossed.

SECTION 06

The Approach & Anomaly Flags

One feature table, herd-relative benchmarks, five domain flags, and a three-class label.

Five anomaly flags drive every downstream decision



Figure — Anomaly flags — three threshold-based, one ratio-based, one window-function-based.

The flags fall into three families. **Threshold** flags (*temp_flag*, *heart_rate_flag*) catch absolute outliers against either a herd benchmark or a fixed physiological range. **Ratio** flags (*rumination_flag*, *low_activity_flag*) catch each cow falling below a configurable share of the herd's hourly average. The **window-function** flag (*sudden_drop_flag*) compares each hour against the previous hour for the same cow using *LAG OVER* (*PARTITION BY cow_id*) and trips when heart rate drops more than 15 bpm or rumination drops more than 30 minutes.

The flags resolve into a three-class *health_status*: **healthy** when all three core flags are false, **at-risk** when temperature or heart rate trips, **unwell** otherwise. This is the label the cow-health classifier learns.

SECTION 07

Code Spotlight

The two SQL fragments that do the most work: sudden-drop detection and the health label.

Sudden-drop detection — earliest predictor of clinical illness

dags/sql/feature_engineering_queries.sql

```
-- hour-over-hour change detection using BigQuery window functions
CASE
  WHEN h.avg_heart_rate - LAG(h.avg_heart_rate) OVER (PARTITION BY h.cow_id ORDER BY
h.hour_ts) < -15
    OR h.avg_rumination_minutes - LAG(h.avg_rumination_minutes) OVER (PARTITION BY h.cow_id
ORDER BY h.hour_ts) < -30
  THEN TRUE
  ELSE FALSE
END AS sudden_drop_flag
```

Health status — the label every Random Forest learns

dags/sql/feature_engineering_queries.sql

```
-- resolve five anomaly flags into a three-class health label
SELECT
  cow_id, hour_ts,
  CASE
    WHEN temp_flag = FALSE AND heart_rate_flag = FALSE
      AND rumination_flag = FALSE THEN 'healthy'
    WHEN temp_flag = TRUE OR heart_rate_flag = TRUE THEN 'at-risk'
    ELSE 'unwell'
  END AS health_status
FROM flags
```

SECTION 08

Machine Learning Models

Three sklearn models, one feature table — health, fertility, and milk-yield forecasting.

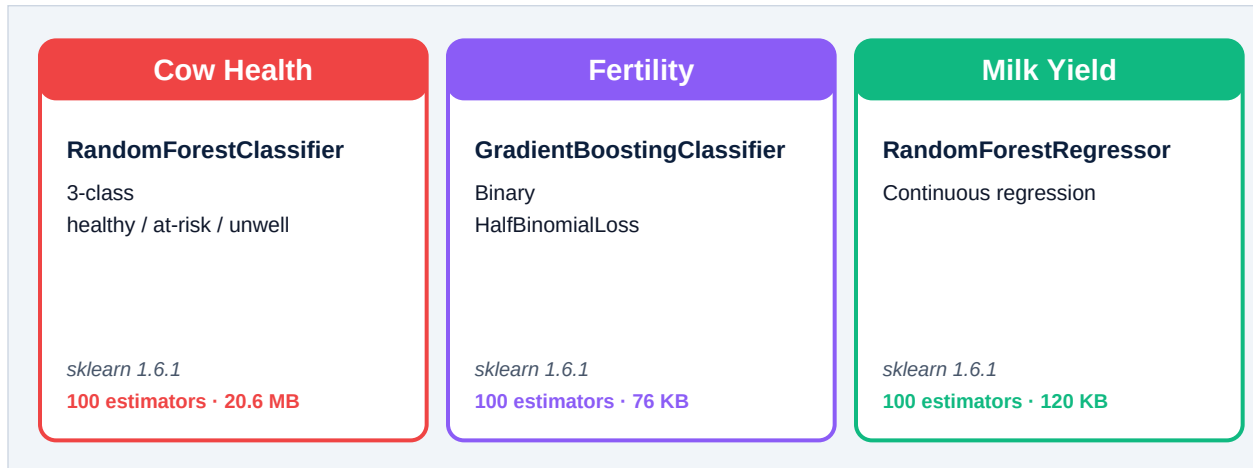


Figure — Three production models, all sklearn 1.6.1, all 100 estimators.

- › **One feature table** — all three models train and score off `smaxtec_features`, eliminating any train/serve drift.
- › **Cow Health classifier** — RandomForestClassifier (100 trees, ~20 MB on disk) predicts the three-class `health_status` label.
- › **Fertility classifier** — GradientBoostingClassifier with HalfBinomialLoss; a binary indicator that ranks each cow on a fertility likelihood.
- › **Milk Yield regressor** — RandomForestRegressor predicts expected milk yield from the same per-cow / per-hour features.
- › **Joblib persistence** — each model is versioned as a single artifact alongside the DAG, so retraining and rollback are file operations, not code changes.
- › **Herd-relative features** — because the inputs benchmark each cow against the herd, the models adapt to seasonal and herd-level shifts without retraining.

SECTION 09

Outcome & Engagement

What the pipeline delivers, and how to take the conversation further.

Smaxtec turns a continuous, high-frequency sensor stream into three actionable signals per cow — current health status, fertility likelihood, expected milk yield — refreshed on a schedule and grounded in herd-aware features. Anomalies trip from rolling benchmarks, not brittle fixed thresholds, so the system adapts to seasonal and herd-level shifts without retraining.

Operationally, the work demonstrates the full data-engineering stack: streaming ingest (Kafka), warehouse modelling (BigQuery), warehouse-side feature engineering (single CTE), classical ML (Random Forest, Gradient Boosting), and orchestrated retraining (Airflow). Every engineered feature lives in one SQL file; every model lives in one joblib artifact; every scheduled run leaves an audit trail.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks Python • Airflow • BigQuery • scikit-learn • Django • Spring Boot